

Bulle d'R. (Re)Découvrir R et RStudio

Sandrine LYSER

22 septembre 2026

Table des matières

1	Préambule	2
2	Présentation des logiciels R et RStudio	2
2.1	R vs RStudio	2
2.1.1	 R	2
2.1.2	 RStudio	4
2.1.3	Les packages	6
3	Travailler avec les projets RStudio et reproductibilité	9
3.1	Qu'est-ce que la reproductibilité en science des données?	9
3.1.1	Comment travailler de façon reproductible sous R et RStudio ?	9
3.1.2	Les projets RStudio	10
3.1.3	Les scripts R	12
4	Introduction au tidyverse	15
4.1	R vs tidyverse	15
4.1.1	R de base	15
4.1.2	tidyverse	15
4.2	Installation du tidyverse	17
4.2.1	Les packages du tidyverse dans la <i>Data Science</i>	17
5	Premières manipulations	17
5.1	Import/Export des données	17
5.1.1	Les fonctions d'import	18
5.1.2	Les fonctions d'export	18
5.2	Manipulations sur les données avec le package <code>[dplyr]</code>	19
5.2.1	Sélection de lignes avec la fonction <code>filter()</code>	19
5.2.2	Sélection de colonnes avec la fonction <code>select()</code>	19
5.2.3	Création de nouvelles variables avec la fonction <code>mutate()</code>	19
5.3	Besoin d'examiner rapidement vos données dans Excel?	19
6	Pour conclure	20
6.1	De nombreuses ressources pour faciliter l'usage de R	20
6.1.1	Des packages et/ou <i>addins</i> (extensions)	20
6.1.2	Des ressources en ligne	20



1 Préambule

📊 Jeu de données utilisé dans cette session

Les exemples s'appuient sur le jeu de données `Journals`, disponible dans le package `Ecdat`

Ce jeu de données se compose de 180 observations décrites par 10 variables

- **title** : titre de la revue
- **pub** : éditeur
- **society** : société
- **libprice** : prix de l'abonnement
- **pages** : nombre de pages
- **charpp** : nombre de caractères par page
- **citestot** : nombre total de citations
- **date1** : année de création de la revue
- **oclc** : nombre d'abonnements
- **field** : domaine de la revue

2 Présentation des logiciels R et RStudio

2.1 R vs RStudio



2.1.1

Langage de programmation et **logiciel libre** pour la statistique, la science des données et la visualisation, développé depuis 1993

- Il ne s'agit pas d'un logiciel "statique" que l'on installe une fois pour des années, c'est tout un écosystème en **perpétuelle évolution, constamment mis à jour et amélioré**
- C'est un logiciel libre distribué selon les termes de la licence GNU GPL (*open source*), qui bénéficie d'une large communauté internationale et qui offre une multitude de fonctionnalités grâce aux packages
- Compatible avec la plupart des systèmes d'exploitation (Windows, MacOS, Linux)
- Disponible en téléchargement sur le site du [CRAN](https://cran.r-project.org/) (*Comprehensive R Archive Network*)

i Dernière version disponible au 21 sept. 2026 : R version 4.6.1 (2026-06-24 ucrt) – "Happy Hop"
Cette session utilise R version 4.6.0 (2026-04-24 ucrt) – "Because it was There"

2.1.1.1 Les objets

La programmation dans **R** repose sur des **objets**, qui apparaîtront dans la fenêtre *Environment* de **RStudio**
⇒ **Dans R, presque tout est un objet : nombres, textes, tableaux, graphiques, modèles, fonctions...**

- Un objet est une structure de données avec
 - un nom (*exemple* : `x`, `donnees`, `modele`)
 - un contenu (une ou plusieurs valeurs)
 - un type / mode (*numeric*, *character*, *logical*, etc.)
 - parfois une classe (*vector*, *matrix*, *data.frame*, *lm*, *ggplot*, etc.)
- On crée un objet avec l'opérateur d'affectation noté `<-`, de la façon suivante : `nom_objet <- valeur`
Exemples

```
x <- 3
nom <- "Alice"
df <- data.frame(age = c(20, 25), sexe = c("F", "H"))
```

Note

- S'il n'existe aucun objet qui porte le nom `nom_objet` dans votre environnement de travail **R**, l'exécution de la commande crée un nouvel objet
- S'il existe déjà un objet avec le nom `nom_objet`, l'exécution de la commande écrase/met à jour l'ancien objet avec la nouvelle valeur

2.1.1.1.1 À quoi servent les objets ?

- À stocker des données (vecteurs de mesures, colonnes d'un tableau, etc.) et des résultats (moyennes, modèles statistiques, tests, prédictions, etc.)
- À structurer l'information : organiser les données en tableaux (*dataframes/tibbles*), listes, matrices, etc.
- À réutiliser du code : les fonctions, graphiques, modèles sont eux-mêmes des objets **R** ⇒ En pratique, presque toutes les commandes **R** créent ou manipulent des objets

Comment nommer les objets ?

Vous pouvez donner n'importe quel nom à vos objets, en respectant certaines règles

- le nom ne peut pas commencer par un chiffre
- le nom ne doit pas contenir de caractères spéciaux (le *underscore* `_` est autorisé)
- le nom ne doit pas être celui d'une fonction **R** déjà existante (*exemple* : `mean`)

2.1.1.1.2 Les vecteurs

Le **vecteur** est le type d'objet de base, le plus simple
Il contient une ou plusieurs valeurs de même type

- du texte - `character` -
- des nombres entiers - `integer` - ou réels - `double` ou `numeric` -
- des valeurs logiques - `TRUE`(vrai) ou `FALSE`(faux) -

Exemples

```
v_log <- c(TRUE, FALSE, TRUE)
v_num <- c(1.5, 2.7, 3.0)
v_txt <- c("rouge", "vert", "bleu")
```

2.1.1.1.3 Les objets plus complexes (1/2)

- Les **dataframes** : des tableaux de données à 2 dimensions, où les lignes sont les observations et les colonnes les variables ⇒ les colonnes peuvent avoir des types différents, mais ont toutes la même longueur

 Les données au format `.csv` ou `.xls` importées sous **R** sont des **dataframes**

Exemple

```
df <- data.frame(id = 1:3,
                 age = c(20, 25, 30),
                 sexe = c("F", "H", "H"),
                 poids = c(55.2, 70.1, 68.4))
```

- Les **matrices** : un objet à 2 dimensions (lignes × colonnes) qui contient des éléments du même type (des données numériques en général)

Exemple

```
M <- matrix(1:6, nrow = 2, ncol = 3)
```

2.1.1.1.4 Les objets plus complexes (2/2)

- Les **listes** : une collection d'objets **R**, qui peuvent être de types différents $\Rightarrow$ une liste peut contenir des vecteurs, des matrices, des *dataframes*, d'autres listes, des fonctions, etc.

Exemple

```
une_liste <- list(age = c(20, 25, 30),  
                 sexe = c("F", "H", "H"),  
                 modele = lm(poids ~ age, data = df))
```

- Les **facteurs** : un format spécial dans **R** pour les variables catégorielles (qualitatives), les modalités sont listées avec *levels* et peuvent être ordonnées (on parle alors d'*ordered factor*)

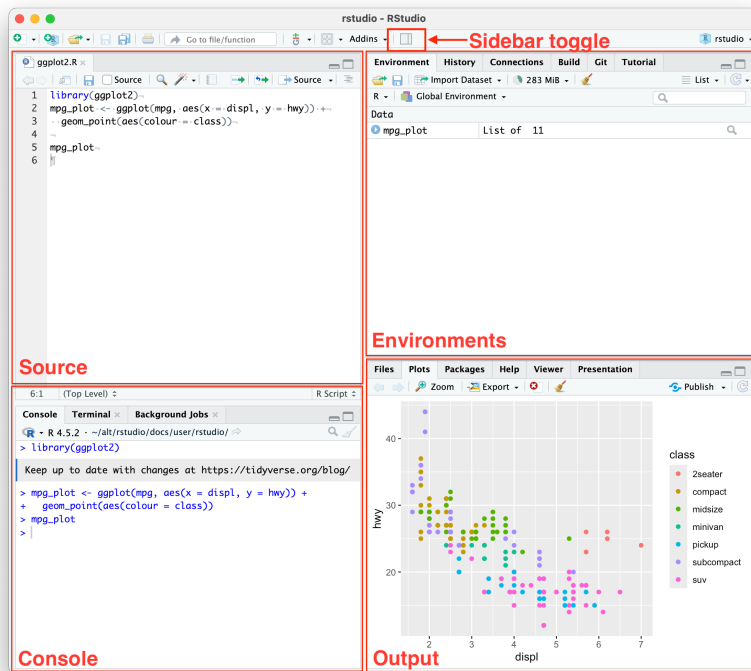
Exemple

```
sexe <- factor(c("F", "H", "F", "H"),  
              levels = c("F", "H"))
```

2.1.2 Studio®

Environnement de développement intégré (IDE) spécialement conçu pour **R** avec une interface conviviale

- Logiciel *open source*, gratuit et disponible pour la plupart des systèmes d'exploitation
- Version locale (*RStudio Desktop*) ou serveur (*RStudio Server*)
- Quatre cadrants pour visualiser facilement les scripts, l'aide, les graphiques
 - **Source** (en haut à gauche) : éditeur de scripts
 - **Environnement** (en haut à droite) : environnement de travail (objets, fonctions, etc.), historique, connexions, tutoriels
 - **Console** (en bas à gauche) : exécution des commandes
 - **Sorties** (en bas à droite) : fichiers, graphiques, packages, aide, visualiseur, présentation



Disposition des volets **RStudio**. Source : [RStudio User Guide Release 2026.08.2](#) - Dernier accès : 2026-09-08

2.1.2.1 1. Les scripts

Des fichiers "texte" pour des traitements qui vont nécessiter des modifications.

Avantages

- garder la trace des lignes de code
- exécuter une/des ligne(s) de code : bouton Run ou raccourci clavier Ctrl + Entrée ou Ctrl + R
- modifier le code
- ajouter des commentaires
- réutiliser le code sur d'autres jeux de données

2.1.2.2 2. L'environnement / L'historique / Les connexions

- Onglet **Environment**
 - objets (données, fonctions, résultats) créés dans cet environnement
 - 2 affichages possibles : liste ou grille (permet de supprimer des objets en cochant la case à côté du nom + clic sur l'icône Balai)
- Onglet **History**
 - historique des commandes exécutées
 - on peut renvoyer le code dans la console ou la source
- Onglet **Connections**
 - pour se connecter à diverses sources de données (par ex. bases de données externes)

2.1.2.3 3. La console

C'est l'équivalent d'une calculatrice

- Sert à effectuer et afficher les résultats de calculs de base (+, -, *, /, etc.)
- Possibilité d'utiliser des fonctions spécifiques : `sum()`, `abs()`, `round()`, etc.

- Les commandes sont exécutées au fur et à mesure qu'elles sont écrites en appuyant sur la touche Entrée

💡 Astuce

On peut naviguer dans l'historique des commandes pour en rappeler une, à l'aide des flèches ↑ et ↓

2.1.2.4 4. Les graphiques/ Les packages / L'aide

- Onglet **Plots**
 - fenêtre où s'affichent les graphiques que l'on crée dans les scripts (ou la console)
 - possibilité de zoomer, de naviguer entre les graphiques, de les copier
 - export des graphiques au format image (jpeg, png, tiff) ou pdf
- Onglet **Packages**
 - liste des packages installés sur la machine
 - installation de nouveaux packages / mise à jour des packages installés
 - un clic sur le nom d'un package affiche les pages d'aide de ce package
- Onglet **Help**
 - afficher la documentation de chaque fonction
 - accéder à des manuels ou des à aides-mémoire (*cheatsheets*)

💡 Comment installer les logiciels ?

L'installation se fait **dans cet ordre**

1. **R** à partir du [CRAN](#) (*Comprehensive R Archive Network*)
2. **RStudio** via le site de [Posit](#)

Le logiciel **R** est régulièrement mis à jour, et contrairement à **RStudio**, il n'y a pas de message d'alerte pour vous prévenir.

💡 **Pensez à vérifier régulièrement et à mettre à jour R pour pouvoir bénéficier des dernières fonctionnalités!**

2.1.3 Les packages

- Bibliothèques de fonctions et de données, développées par la communauté
- Améliorent les fonctionnalités de base de **R** ou proposent des fonctionnalités supplémentaires
- Contiennent de la documentation sur les fonctions qu'ils proposent

Exemple : {ggplot2}, {dplyr}, {graphics}

2.1.3.1 Installer un package

L'installation d'un package dépend de l'endroit où les auteurs l'ont déposé

- Si le package est sur le CRAN, l'installation se fait avec la fonction `install.packages()`

```
# Installer le package ggplot2
install.packages("ggplot2")
```

- Si les packages sont sur des dépôts, l'installation se fait grâce au package [remotes](#), avec des fonctions qui dépendent du dépôt
 - GitHub : fonction `install_github()`
 - GitLab : fonction `install_gitlab()`

```
# Installer Le package viewxl qui se trouve sur un dépôt Github
remotes::install_github("dreamRs/viewxl")
```

💡 L'installation ne doit pas être refaite à chaque exécution du script!

2.1.3.2 Charger un package

Cette étape est nécessaire à chaque fois que l'on veut utiliser une ou plusieurs fonctions d'un package

```
# Charger Le package ggplot2
library(ggplot2)
```

⚠ La gestion des packages

Les packages évoluent, les fonctions changent et avec une nouvelle version de package, votre code peut ne plus s'exécuter sans erreur

💡 **Il est vivement recommandé de lister les packages utilisés au début de vos scripts, pour les identifier plus rapidement, en notant leur version**

— "à la main"

```
library(tidyverse)    # v2.0.0
library(questionr)    # v0.8.2
```

— de façon automatique

```
library(tidyverse)
library(questionr)
```

```
packages_utilises <- sessionInfo()
```

2.1.3.3 Que se passe-t'il quand on installe une nouvelle version de R?

- Seul les packages de base sont installés, dans leur version mise à jour
- Les packages qui ont été installés en fonction des besoins ne le sont pas, il faut les réinstaller

i **RStudio** affiche un avertissement en haut des scripts qui appellent des packages qui ne sont pas installés

2.1.3.3.1 Comment réinstaller les packages automatiquement?

Les étapes à suivre

- Créer un script **R** qui va contenir la liste des packages à installer, quel que soit le projet dans lequel vous travaillez
 - pour les packages qui se trouvent sur le CRAN, lister les packages dans la commande `install.packages()`
 - pour les packages qui se trouvent sur les dépôts GitHub, l'installation se fera avec la commande `install_github()` du package [remotes](#)
- Mettre à jour le script au fur et à mesure des installations de nouveaux packages
- Exécuter le contenu du script, en ayant pris soin auparavant d'actualiser la version de **R** sous **RStudio** si plusieurs sont installées!

Exemple de script

```
# _____  
# Script: Packages à installer après chaque nouvelle installation de R  
# Author: Sandrine.Lyser  
# Créé Le : 2026-06-19  
# Dernière modification : 2026-09-07  
# Text default encoding : UTF-8  
# _____  
  
# Packages du CRAN  
install.packages(c("tidyverse", "esquisse", "RColorBrewer", "remotes", "maps"),  
                 dependencies = TRUE)  
  
# Autres packages (sur github par exemple)  
remotes::install_github("lorenzwalther/strcode", dependencies = TRUE)  
  
remotes::install_github("dreamRs/viewxl", dependencies = TRUE)
```

i Le préfixage

La notation

```
dplyr::filter(Journals, field == "Econometrics")
```

peut remplacer le code

```
library(dplyr)  
filter(Journals, field == "Econometrics")
```

Avantages

- Évite les conflits si plusieurs packages ont une fonction avec le même nom (mais des paramètres différents)
Exemple la fonction `filter()` existe dans les packages `{stats}` et `{dplyr}`
- Identifie le package d'où provient une fonction
- Ne charge pas la totalité d'un package pour n'utiliser qu'une seule de ses fonctions

💡 Vous ne savez pas comment faire pour citer **R** ainsi que les packages que vous avez utilisés dans vos travaux ?

La fonction `citation()` est là pour vous aider !

- **Pour citer la version de R**

```
# Citer R  
citation()
```

To cite R in publications use:

R Core Team (2026). *_R: A Language and Environment for Statistical Computing_*. R Foundation for Statistical Computing, Vienna, Austria.
doi:10.32614/R.manuals <<https://doi.org/10.32614/R.manuals>>.
<<https://www.R-project.org/>>.

Une entrée BibTeX pour les utilisateurs LaTeX est

```
@Manual{,
  title = {R: A Language and Environment for Statistical Computing},
  author = {{R Core Team}},
  organization = {R Foundation for Statistical Computing},
  address = {Vienna, Austria},
  year = {2026},
  doi = {10.32614/R.manuals},
  url = {https://www.R-project.org/},
}
```

We have invested a lot of time and effort in creating R, please cite it when using it for data analysis. See also 'citation("pkgname")' for citing R packages.

— Pour citer un package

```
# Citer un package (ici ggplot2)
citation("ggplot2")
```

To cite ggplot2 in publications, please use

H. Wickham. ggplot2: Elegant Graphics for Data Analysis.
Springer-Verlag New York, 2016.

Une entrée BibTeX pour les utilisateurs LaTeX est

```
@Book{,
  author = {Hadley Wickham},
  title = {ggplot2: Elegant Graphics for Data Analysis},
  publisher = {Springer-Verlag New York},
  year = {2016},
  isbn = {978-3-319-24277-4},
  url = {https://ggplot2.tidyverse.org},
}
```

3 Travailler avec les projets RStudio et reproductibilité

3.1 Qu'est-ce que la reproductibilité en science des données ?

= Processus qui peut être reproduit par une autre personne et/ou à partir des mêmes données mises à jour et qui permet d'expliquer et partager son travail

S'appuie sur **trois grands principes** interagissant entre eux et non séquentiels¹

1. Organiser son travail
2. Coder pour les autres
3. Automatiser le plus possible

3.1.1 Comment travailler de façon reproductible sous R et RStudio ?

1. En utilisant les **projets RStudio** : on isole chaque analyse
2. En stockant ses traitements dans des **scripts R** : on garde une trace des commandes, pas de travail dans la console
3. En utilisant les **chemins relatifs** : pour exécuter sans problème le code sur un autre ordinateur
4. En prêtant une attention à la **gestion des packages** : versions et dépendances

1. Orozco, V., C. Bontemps, E. Maigné, V. Piguet, A. Hofstetter, A. Lacroix, F. Levert, and J-M. Rousselle. 2020. "How To Make A Pie : Reproducible Research for Empirical Economics and Econometrics." Journal of Economic Surveys 34 (5) : 1134–69. <https://doi.org/10.1111/joes.12389>.

5. En **documentant** son travail : commentaires, fichier README

i Zoom sur l'encodage

L'encodage est un processus de conversion d'informations (texte, données, etc.) en un format lisible, compréhensible et exploitable par différents systèmes ⇒ il permet une compatibilité entre systèmes

- Il existe plusieurs encodages, comme par exemple : ASCII, ISO 8859-15, UTF-8
- Les encodages sont différents selon le système d'exploitation (Windows-1252 sous Windows, UTF-8 sous Linux ou Mac)

💡 **Il est recommandé d'utiliser l'encodage UTF-8 pour les scripts et les données**, car c'est à l'heure actuelle l'encodage le plus universel

⚙️ **Comment peut-on définir l'encodage sous RStudio ?**

- Allez dans Tools > Global Options > Code > Saving > Default text encoding
- Sélectionnez UTF-8

💡 La fonction `set.seed()`

Automatiser c'est aussi pouvoir refaire les mêmes calculs et obtenir les mêmes résultats

⇒ utiliser la fonction `set.seed()` permet de garantir la reproductibilité des résultats de tout travail inscrit dans un script **R** qui implique de l'aléatoire (simulation permutation, bootstrap, etc.)

💡 **Placée avant une opération aléatoire (fonctions `sample()`, `rnorm()` par exemple), elle permet de retrouver les mêmes résultats à chaque exécution**

```
set.seed(123)      # 'Graine' (valeur quelconque) pour le générateur aléatoire
sample(1:10, 5)    # Un tirage reproductible
```

```
[1] 3 10 2 8 6
```

3.1.2 Les projets RStudio

Les projets **RStudio** sont utiles pour

- **organiser plus facilement son travail** : 1 projet **RStudio** = 1 projet de recherche / 1 étude / 1 cours, etc.
⇒ environnement isolé, pas de conflits entre projets
- **regrouper dans un même dossier tous les scripts** : scripts **R**/quarto, fonctions, données, résultats, graphiques, etc.
- **améliorer la portabilité et faciliter les collaborations** : utilisation des chemins de fichiers relatifs ⇒ reproductibilité facilitée

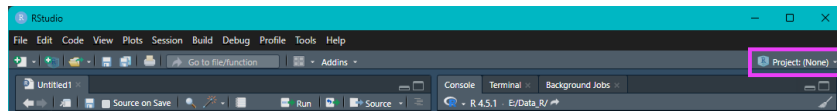
i Chemins absolus/relatifs

- Le chemin absolu est exprimé par rapport au disque dur, à l'ordinateur
Exemple : C:/Users/...
- Le chemin relatif est exprimé par rapport au projet
Exemple : data/mon_fichier.csv

💡 **Le répertoire par défaut du projet est le dossier où est enregistré le projet sur votre machine**

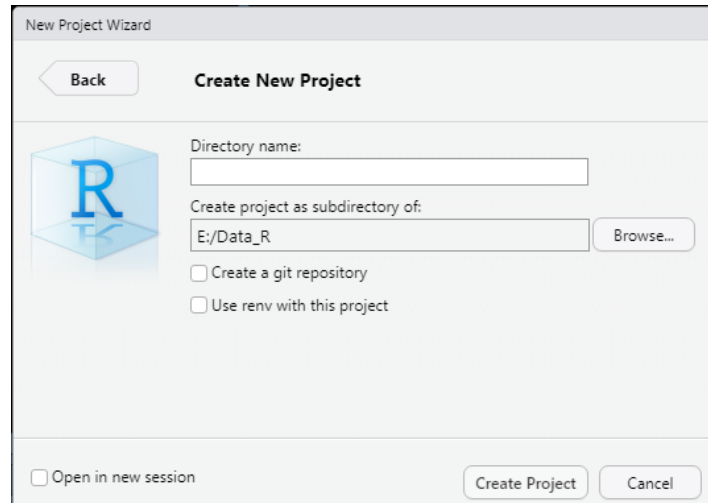
3.1.2.1 Comment créer un projet ?

À partir de l'icône dédiée en haut à droite de **RStudio**



Cliquez sur New Project > New Directory > New Project et complétez les informations requises

- le nom du dossier/projet
- son emplacement (dossier de votre choix)



💡 Quand le projet est créé

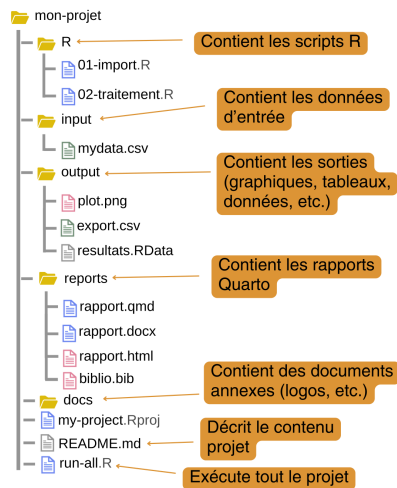
- **RStudio** redémarre et se place dans ce projet
- le projet apparaît dans l'onglet *Files* de **RStudio**
- 💡 Pour ranger efficacement les différents documents liés à l'analyse, vous pouvez créer les sous-dossiers
 - soit en cliquant sur **New Folder** dans le menu en haut de l'onglet *Files*
 - soit de façon classique, à partir de l'explorateur de fichier de votre machine

3.1.2.2 Comment structurer ses projets ?

Choisissez la structuration qui vous correspond le mieux pour **rassembler** tous les éléments liés au projet dans des dossiers/sous-dossiers qui séparent données source/créées, scripts, documentation, résultats, etc.

- 💡 **Chacun s'organise de la façon qu'il juge la plus adaptée à ses usages, en gardant en tête que ce sont les données initiales et les scripts de traitement qui sont les plus importants pour pouvoir rejouer les analyses**

ARBORESCENCE DE PROJET



```

name_of_project
|--data
|  |--2017report.csv
|  |--2016report.pdf
|  |--summary2016_2017.csv
|--docs
|  |--01-analysis.Rmd
|  |--01-analysis.html
|--scripts
|  |--exploratory_analysis.R
|--name_of_project.Rproj
|--run_all.R
  
```

```

name_of_project
|--raw_data
|  |--WhateverData.xlsx
|  |--2017report.csv
|  |--2016report.pdf
|--output_data
|  |--summary2016_2017.csv
|--rmd
|  |--01-analysis.Rmd
|--docs
|  |--01-analysis.html
|  |--01-analysis.pdf
|  |--02-deeper.html
|  |--02-deeper.pdf
|--scripts
|  |--exploratory_analysis.R
|  |--pdf_scraper.R
|--name_of_project.Rproj
|--run_all.R
  
```

Exemples de structuration de projets **R** Source (figures 2 & 3) : <https://learn.r-journalism.com/en/publishing/workflow/r-projects/> - Dernier accès : 2026-09-08

3.1.3 Les scripts R

- Ils permettent de **garder la trace** du processus de création de chaque variable, tableau, figure, etc., ce qui permet de refaire les traitements dans leur globalité
- Ils sont **facilement partageables** avec d'autres, sous réserve de respecter des pratiques de "bon usage"
 - coder de façon compréhensible avec des commentaires et en documentant ses fonctions
 - donner des noms explicites aux objets et fonctions
 - utiliser les conventions de nommage, notamment pour distinguer les variables d'origine des variables créées



- Pour les **variables**, utiliser un **nom**
Exemple : temperature_max
- Pour les **fonctions**, utiliser un **verbe**
Exemple : create.map serait une fonction qui permettrait de créer une carte
- Pour **distinguer variables d'origine / variables recodées**, on peut utiliser un préfixe ou un suffixe
Exemple : age est la variable d'origine et age_recod est la variable recodée

i Les conventions de nommage

Il existe différentes conventions

- **alllowercase** : tout en minuscule, sans séparateur
- **period.separated** : tout en minuscule, mots séparés par des points
- **underscore_separated** : tout en minuscule, mots séparés par un *underscore* (`_`)
- **lowerCamelCase** : première lettre des mots en majuscule, à l'exception du premier mot ; et si nom simple, tout en minuscule
- **UpperCamelCase** : première lettre des mots en majuscule, y compris le premier et même lorsque le nom est composé d'un seul mot

💡 **Possibilité de mixer les conventions mais en gardant une cohérence afin de faciliter la compréhension des fichiers et du code**

⚠ **R est sensible à la casse** ce qui signifie que `variable` et `VariabLe` sont deux objets différents

3.1.3.1 Comment organiser ses scripts ?

- Prévoir un **pipeline analytique**

= *séquence ordonnée de traitements automatisés permettant de transformer, analyser et visualiser des données de façon reproductible et efficace*

- import et préparation des données (nettoyage, fusion, filtrage)
- traitement statistique (modélisation, transformation, calculs)
- production des résultats (graphiques, tableaux, rapports)
- Utiliser une convention de dénomination des fichiers et leur donner des noms explicites

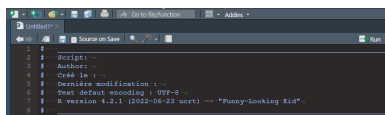
💡 Pour les **fichiers**, on peut les nommer avec la structure `numero_nom_millesime.extension`, sans symboles spéciaux dans le nom
Exemple : `01_import_donnees_20250606.R`
Les **caractères spéciaux** qui sont à proscrire sont `'- . , ; : \ / $ ^`, ainsi que les caractères accentués

i Ressources sur le processus d'analyse et de communication des données reproductibles

 R Workflow
 Reproducible analytical pipeline

3.1.3.2 Comment structurer ses scripts ?

1. Commencer l'écriture du script par les **métadonnées** (titre, auteur, date, version de **R**, encodage, etc.)



2. Ajouter la **liste des packages utilisés** et leur version
3. **Structurer** en parties et sous-parties grâce à l'*addin strcode*
 `remotes::install_github("LorenzWalthert/strcode")`

```
# _____  
# Titre 1 #####  
## .....  
## Titre 2 #####
```

4. Commenter les scripts (avec le signe #), en particulier pour expliquer les étapes-clés

💡 Comment passer une ligne entière en commentaires ?

💡 **Avec un raccourci clavier** : se placer sur la ligne et cliquer sur `CRTL + MAJ + C`

i Respecter des règles pour faciliter la lecture

- Retour à la ligne régulièrement (**la longueur maximale recommandée d'une ligne est 80 caractères**) et après chaque pipe (`%>%` ou `|>`)

- L'indentation se fait grâce à la touche TAB. Le raccourci Ctrl+I sert à réindenter les lignes de code sélectionnées
- Les opérateurs qui lient les objets entre eux (=, +, -, <-, etc.) sont entourés d'espaces
- Les opérateurs qui modifient un objet ou sélectionnent une partie d'un objet (:, !!, \$, @, [,], [[,]]) ne sont pas entourés d'espaces

```
# On écrit
1 + 1 = 2
dataframe$variable
```

```
# et non
1+1=2
dataframe $ variable
```

- On insère un espace après la virgule mais pas avant, comme en français!
- On ne met pas d'espace avant ou après les parenthèses
- En revanche, les opérateurs de comparaison doivent être entourés d'espace pour ne pas être mal interprétés par R

```
# On écrit
mean(x, na.rm = TRUE)
1 < -10
```

```
# et non
mean ( x , na.rm = TRUE )
1<-10
```

💡 Aide pour écrire les scripts

⚙️ Activer les options de diagnostic pour que **RStudio** signale automatiquement les erreurs dans les scripts: Tools > Global Options > Code > Diagnostics

```
6 #
7 # Packages #####
8 library(tidyverse) # v 2.0.0
9
```

📦 package [\[styler\]](#) : mettre en forme le code selon le [Tidyverse Style Guide](#) (ou selon son propre style)

📦 package [\[lintr\]](#) : vérifier que le code est conforme à un guide de style donné

💡 Le fichier README

Il est recommandé de créer un fichier README, dans le dossier du projet, qui présente le **projet**, sa **structuration** et les **étapes d'exécution des différents script** pour comprendre, reproduire et utiliser le projet

💡 Un README est généralement un fichier au format .txt ou .md (markdown)

Exemple de fichier README

```

# BulleR

Projet de la session

## Structure
- data/ : données
  └─ sources/ : données sources, données brutes
  └─ produites/ : données produites
- scripts/ : scripts R, qmd
- outputs/ : graphiques et tableaux exportés
- docs/ : rapports et supports

## Comment reproduire les analyses
1. Ouvrir le projet `BulleR`
2. Exécuter les scripts dans l'ordre (01_import.R, 02_nettoyage.R, etc.)
3. Les graphiques générés sont enregistrés dans le dossier outputs
4. Le rapport d'étude est enregistré dans le dossier docs

```

4 Introduction au tidyverse

4.1 R vs tidyverse

- Deux "façons" d'écrire du **R**
- Mêmes résultats, styles différents

4.1.1 R de base

- Fonctions et syntaxe incluses dans **R**
- Syntaxe parfois hétérogène
- Beaucoup d'opérateurs spéciaux pour manipuler les données : `[`, `[[`, `$`, `which()`, `apply()`, etc.
- Les fonctions s'imbriquent, création de nombreux objets intermédiaires
- Code souvent "compact", mais parfois difficile à lire pour les débutants

Exemple

```

res <- head(subset(Journals, citestot > 20))
# ou
tmp <- Journals[Journals$citestot > 20, ]
res <- head(tmp)

```

4.1.2 tidyverse

- Ensemble de packages qui reprennent des opérations courantes de **R** de façon unifiée, conçus pour travailler ensemble
 - uniformité des formats d'entrée et de sortie
 - syntaxe commune entre les packages
 - les noms de fonctions sont des verbes : `filter()`, `select()`, `mutate()`, `summarise()`, `arrange()`, `join()`
- Philosophie "tidy" : données en tables "propres", fonctions cohérentes, code lisible
- L'accent est mis sur la lisibilité et la chaîne d'opérations, via l'opérateur *pipe* `|>`
 - ⇒ le code suit l'ordre logique des étapes, avec moins d'objets intermédiaires, moins de parenthèses imbriquées

Exemple

```

library(dplyr)

```

```
Journals |>
  filter(citestot > 20) |>
  head()
```

4.1.2.1 L'opérateur pipe

- Opérateur qui permet d'enchaîner des opérations : il prend la sortie d'une fonction (ou d'une expression) et la passe automatiquement comme **premier** argument à la fonction suivante
- Initialement fourni dans le package [magrittr](#) et noté %>%
- Implémenté dans le langage **R** de base depuis la version 4.1.0 et noté |>

```
data |>
  fonction1() |>
  fonction2() |>
  fonction3()
# équivaut à
fonction3(fonction2(fonction1(data)))
```

💡 Il est recommandé d'utiliser le pipe natif de R |>

⚙️ Comment utiliser le pipe natif sous RStudio ?

- Allez dans Tools > Global Options > Code > Editing
- Cochez Use native pipe operator, |> (requires R 4.1+)

4.1.2.2 Les tibbles

- **Version modernisée des *dataframes***, proposée dans le cadre du tidyverse
- Plus pratiques à utiliser que les *dataframes* "classiques"
- Plus rapides à manipuler (et plus stables) que les *dataframes*
- Il est possible de convertir d'un type à l'autre
 - *dataframe* → *tibble* : avec la fonction `as_tibble()` du package [tibble](#)
 - *tibble* → *dataframe* : avec la fonction **R** de base `as.data.frame()`

i Aperçu des formats

dataframe

```
Journals |>
  head(2)
```

	title	pub	society	libprice	pages	charpp	citestot
1	Asian-Pacific Economic Literature	Blackwell	no	123	440	3822	21
2	South African Journal of Economic History So Afr ec history assn		no	20	309	1782	22

tibble

```
Journals |>
  as_tibble() |>
  head(2)
```

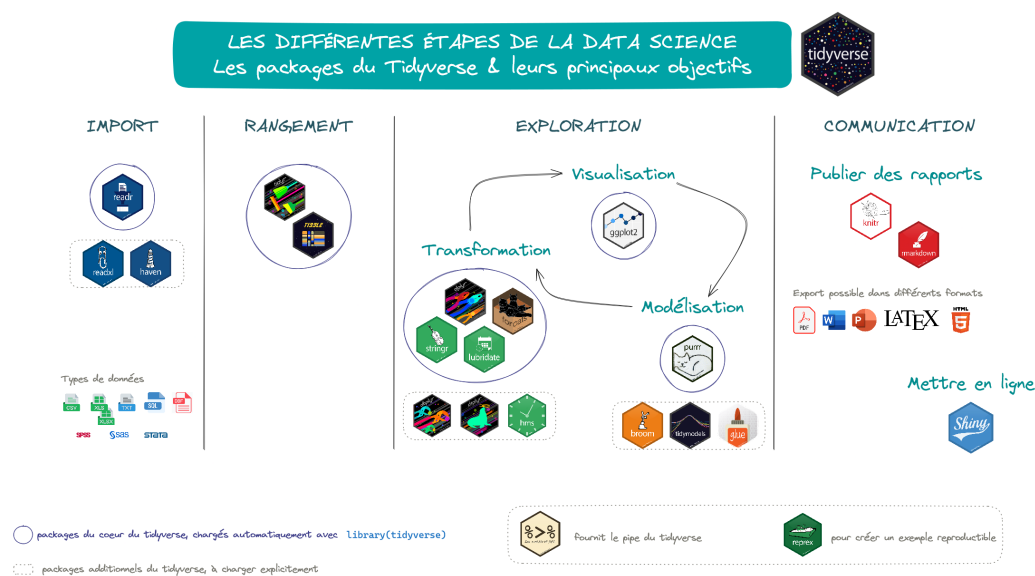
A tibble: 2 x 10

	title	pub	society	libprice	pages	charpp	citestot
	<fct>	<fct>	<fct>	<int>	<int>	<int>	<int>
1	Asian-Pacific Economic Literature	Blackwell	no	123	440	3822	21

4.2 Installation du tidyverse

- ✨ La commande `install.packages("tidyverse")` installe simultanément plusieurs packages
 - `{dplyr}` : manipulation des données
 - `{forcats}` : traitement des variables qualitatives
 - `{ggplot2}` : visualisation des données
 - `{lubridate}` : manipulation des dates et heures
 - `{purrr}` : programmation
 - `{readr}` : import de données
 - `{stringr}` : manipulation des chaînes de caractères
 - `{tibble}` : version modernisée des *dataframes*, plus pratiques à utiliser que les *dataframes* "classiques"
 - `{tidyr}` : nettoyage, remise en forme des données

4.2.1 Les packages du tidyverse dans la Data Science



5 Premières manipulations

5.1 Import/Export des données

- Depuis/vers des formats "plats" ou délimités (.csv, .txt)
- Depuis/vers des fichiers Excel ou issus d'autres logiciels de statistique (Stata, SPSS, etc.)
- Depuis/vers des bases de données relationnelles

💡 Pour l'import, bien étudier le jeu de données source, à l'aide des questions suivantes

1. La première ligne contient-elle le nom des variables ?
2. Quel est le séparateur des colonnes : , , ; , espace(s), \t ?
3. Quel est le caractère utilisé pour indiquer les décimales : le point (à l'anglo-saxonne) ou la virgule (à la française) ?
4. Les valeurs textuelles sont-elles encadrées par des guillemets ? Si oui, s'agit-il de guillemets

simples (') ou de guillemets doubles (")?

5. Y a-t-il des valeurs manquantes ? Si oui, comment sont-elles indiquées ?

5.1.1 Les fonctions d'import

Type de fichier	Séparateur de colonnes	Séparateur décimal	Tidyverse	Base R
Délimité	,	.	<code>readr::read_csv()</code>	<code>read.csv()</code>
	;	,	<code>readr::read_csv2()</code>	<code>read.csv2()</code>
	un espace (ou plus)	.	<code>readr::read_table()</code>	<code>read.table()</code>
	\t	.	<code>readr::read_delim()</code>	<code>read.delim()</code>
	\t	,	<code>readr::read_delim2()</code>	<code>read.delim2()</code>
Excel			<code>readxl::read_excel()</code>	<code>xlsx::read.xlsx()</code>
SPSS			<code>haven::read_spss()</code>	<code>foreign::read.spss()</code>
			<code>haven::read_sav()</code>	
Stata			<code>haven::read_stata()</code>	<code>foreign::read.dta()</code>
SAS			<code>haven::read_sas()</code>	<code>foreign::read.ssd()</code>
dBase				<code>foreign::read.dbf()</code>

5.1.2 Les fonctions d'export

Type de fichier	Tidyverse	Base R
.txt	<code>readr::write_delim(delim = "\t")</code>	<code>write.table()</code>
.csv	<code>readr::write_csv()</code>	<code>write.csv()</code>
.xlsx	<code>openxlsx::write_xlsx()</code> <code>writexl::write_xlsx()</code>	
.dbf		<code>foreign::write.dbf()</code>
.sav (SPSS)	<code>haven::write_sav()</code>	<code>foreign::write.foreign(package = "SPSS")</code>
.dta (Stata)	<code>haven::write_dta()</code>	<code>foreign::write.dta()</code>

5.1.2.0.1 Exemple d'import pour un fichier .csv

```
readr::read_csv("file.csv"  
  # si la première ligne contient le nom des variables :  
  , col_names = TRUE)
```

💡 Zoom sur la fonction read_csv2()

La fonction `readr::read_csv2()` considère par défaut que le séparateur de colonnes est le point-virgule et que la virgule sert de séparateur décimal

La fonction en langage R de base `read.csv2()` a les mêmes paramètres par défaut

💡 **Utiliser ces fonctions lorsque les fichiers .csv ont ces caractéristiques**

5.2 Manipulations sur les données avec le package {dplyr}

5.2.1 Sélection de lignes avec la fonction filter()

```
# Revues dans Le domaine de La Démographie
Journals |>
  dplyr::filter(field == "Demography")
```

	title	pub	society	libprice	pages	charpp	citestot	date1	oclc	
30	Journal of Population Economics	Springer	no	411	640	3264	69	1987	27	Demog
116	Population & Development Review	Population Council	no	36	910	2904	445	1975	218	Demog
137	Demography	Pop Assn America	no	85	568	6859	670	1964	413	Demog

5.2.2 Sélection de colonnes avec la fonction select()

```
# Sélection de la variable "pub"
Journals |>
  dplyr::select(pub)
```

	pub
1	Blackwell
2	So Afr ec history assn
3	Kluwer
4	Kluwer
5	Elsevier
6	Elsevier
7	Cambridge Univ Pres
8	Elsevier
9	Kluwer
...	

5.2.3 Création de nouvelles variables avec la fonction mutate()

```
# Ajout de la variable "anciennete"
Journals |>
  mutate(anciennete = 2025 - date1) -> Journals

names(Journals)
```

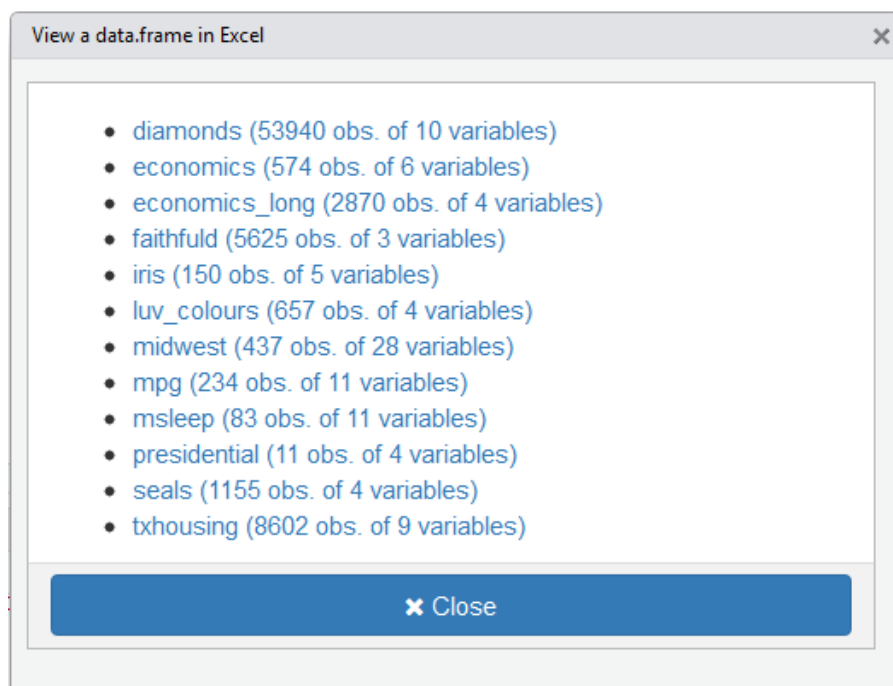
[1]	"title"	"pub"	"society"	"libprice"	"pages"	"charpp"	"citestot"	"date1"
[9]	"oclc"	"field"	"anciennete"					

💡 Penser à affecter le résultat à un objet si nécessaire!

5.3 Besoin d'examiner rapidement vos données dans Excel ?

Utilisez l'addin viewxl pour exporter interactivement les *dataframes* de l'environnement **R** vers Excel

— via le menu Addins > VIEWXL > View in Excel



— dans la console, avec la fonction `view_in_xl()` :

```
viewxl::view_in_xl(diamonds)
```

⚠ viewxl ne sert qu'à visualiser les données sous Excel, et non pas à faire des modifications sur les données. Les modifications se font toujours sur R!

6 Pour conclure

6.1 De nombreuses ressources pour faciliter l'usage de R

6.1.1 Des packages et/ou *addins* (extensions)

 [\[addinslist\]](#) : parcourir et installer les *addins* de **RStudio**

 [\[questionr\]](#) : manipuler, recoder, nettoyer et analyser (tableaux de fréquence par exemple) des données d'enquêtes (par questionnaire)

 [\[ReplaceInFiles\]](#) : rechercher et remplacer une valeur dans plusieurs fichiers

 [\[strcode\]](#) : structurer le code (sauts de section avec titre en en-tête)

6.1.2 Des ressources en ligne

 Sur le CRAN, des [manuels](#) et d'[autres contributions](#)

 [R project](#)

 [Rseek](#) : moteur de recherche dédié à **R**

 Des [cheatsheets](#) (fiches synthétiques) pour certains packages (accessibles également via la page d'accueil de la fenêtre d'aide de **RStudio**)

 [frrrenchies](#) : liste de ressources francophones

 [StackOverflow](#)

 [R journal](#)

 [Journal of Statistical Software](#)

6.1.2.1 Sans oublier... la fiche de "bonnes pratiques" à ETTIS

 [Cheatsheet Bonnes Pratiques en R à ETTIS](#) (PDF)